

Foundations Continued: Pretraining / Post Training

CSE 5539: Special Topics in Large Language Models

<https://shocheen.github.io/courses/cse-5539-fall-2026>



THE OHIO STATE UNIVERSITY

Logistics

- Foundations homework is due tonight, 11.59pm.
- Project proposal due Sept 25.
 - Will release more details tomorrow.

Recap

- What is a language model
- Neural language models
- Attention and Transformers
- Generating from a transformer-based language model
- Computational consideration in transformers
- Pretraining data

Today: a brief overview of the modern language modeling pipeline

- Pretraining
- Post-training or “Alignment”
 - SFT
 - Instruction tuning
 - Main paper presentation with roles
 - RLVR
 - RLHF (depending on time)

Pretraining

Train a randomly initialized model on a large text corpora

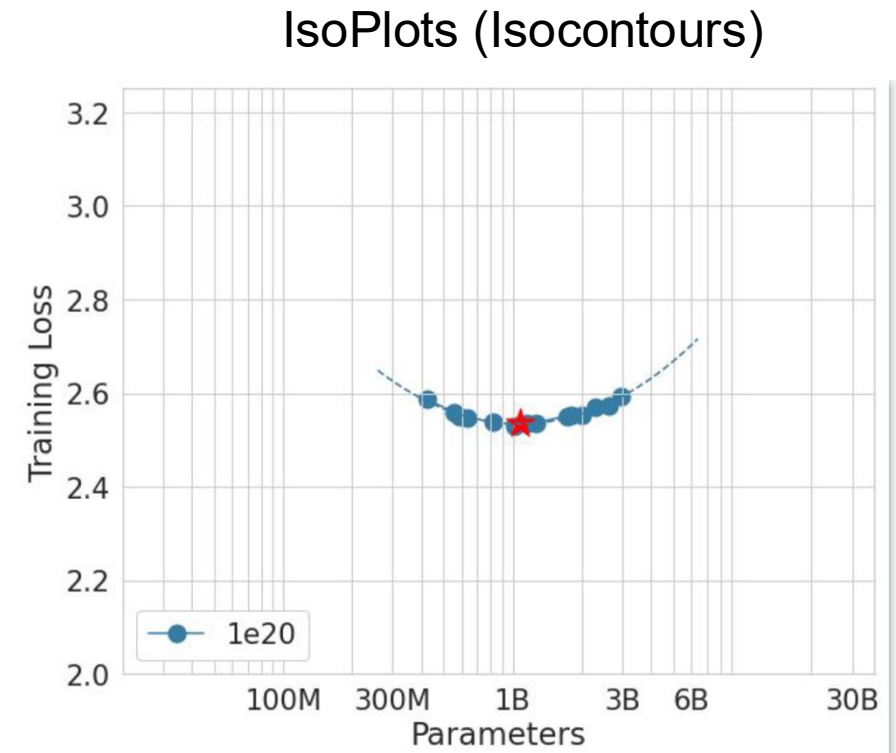
How should we select the right model size, data size, batch size etc (hyperparameters) for pretraining?

Q: What would you do?

- Zuckerberg gave you a \$500M budget for training Muse Spark 3.
- You set aside \$10M for finding the best architecture at smaller scale, assuming that your ultimate model will be much larger.
- This way, you pick your parameters with rigorous experiments at small scale:
 - Optimal training params: Learning rate, warmup, weight decay, etc.
 - Architecture configs by scaling (each x50) the optimal values at small scale.
- Q: What you like (or don't) about this recipe?
- Optimal depth/width, lr, batch size, weight decay, init, and residual scaling are **not scale-invariant**.

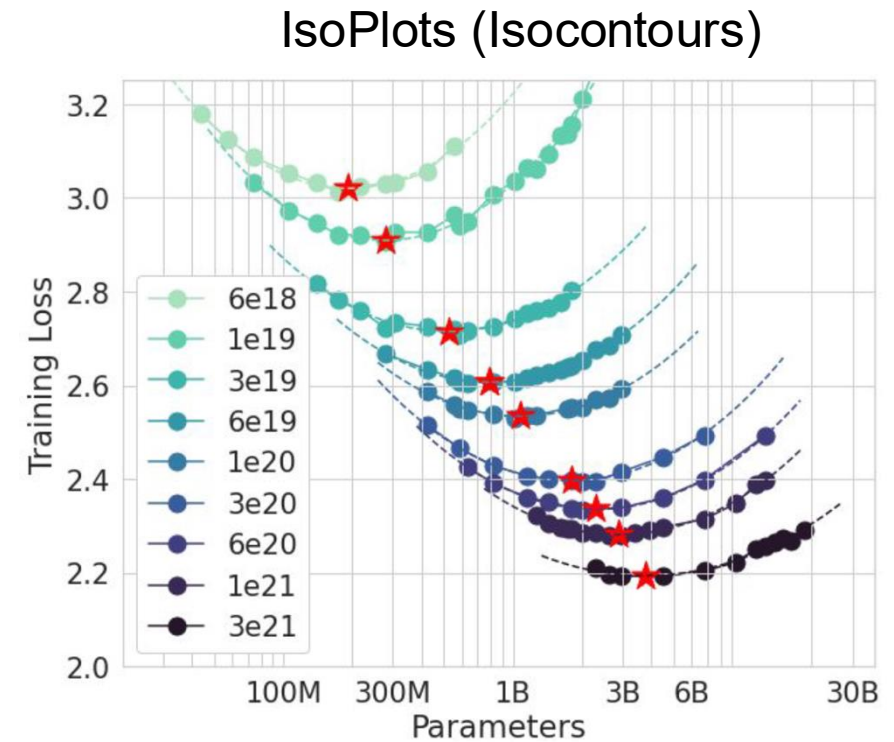
IsoPlots: Tradeoffs at a smaller scale

- The performance of your model depends on a complex combination of many factors.
- Goal: find the best combinations, for a fixed compute.
- Approach:
 1. Fix a compute budget FLOP
 2. Train a few models and vary their size
 3. Fit a curve and find the minimum



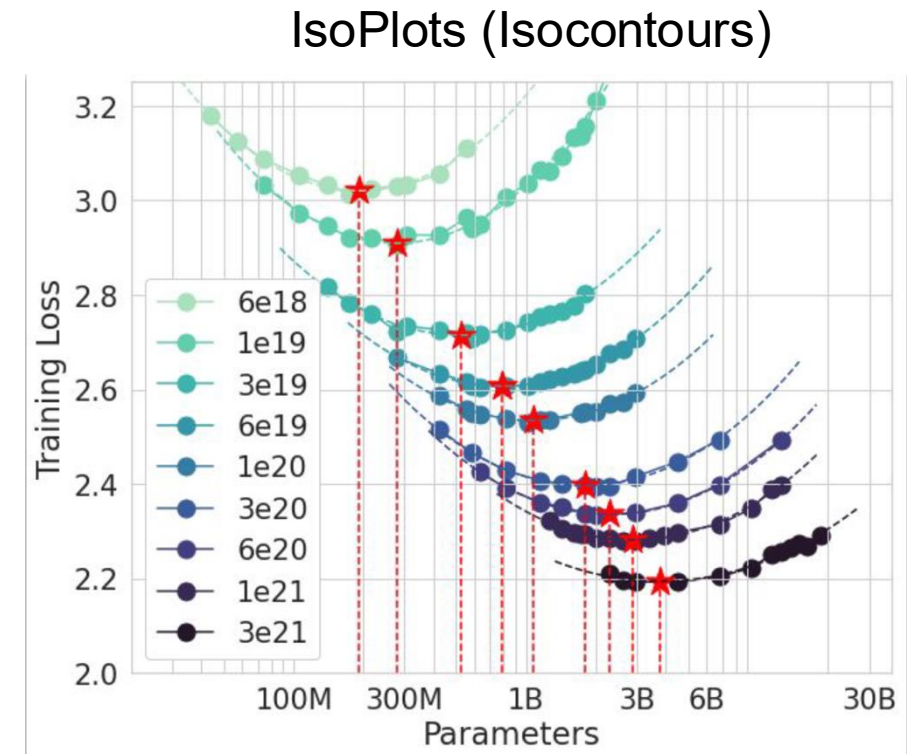
IsoPlots: Tradeoffs at a smaller scale

- The performance of your model depends on a complex combination of many factors.
- Goal: find the best combinations, for a fixed compute.
- Approach:
 1. Fix a compute budget FLOP
 2. Train a few models and vary their size
 3. Fit a curve and find the minimum
 4. Repeat 1-3 for various FLOP budgets



IsoPlots: Tradeoffs at a smaller scale

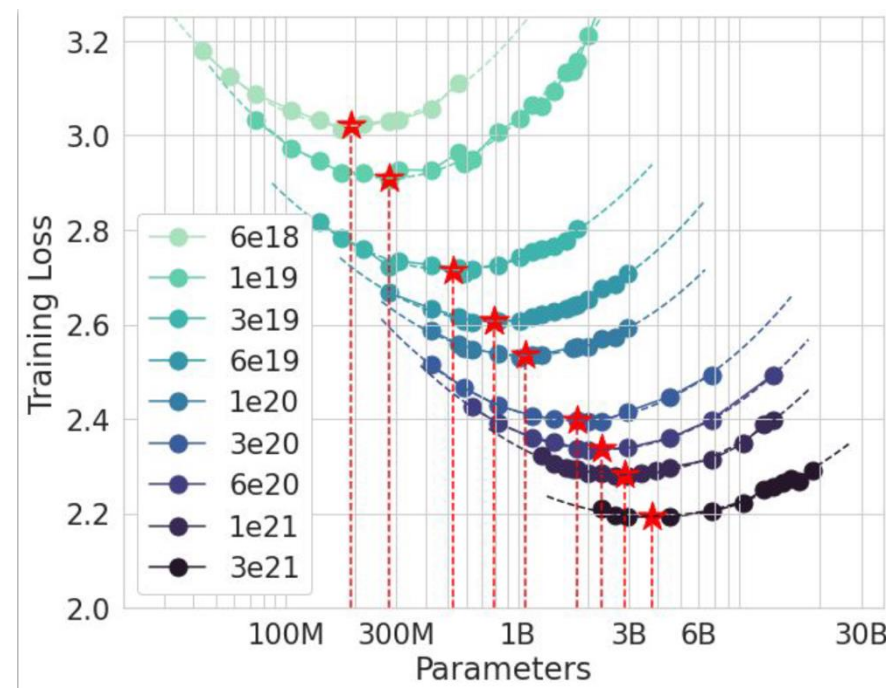
- The performance of your model depends on a complex combination of many factors.
- Goal: find the best combinations, for a fixed compute.
- Approach:
 1. Fix a compute budget FLOP
 2. Train a few models and vary their size
 3. Fit a parabola and find the minimum
 4. Repeat 1-3 for various FLOP budgets
- It's good to change various parameter (e.g., training data, size, or other hyperparams) and see how it's quality (loss) changes.



Predictive models for parameters

- Overall, IsoPlots show how loss depends on three axes: model size (parameters N), dataset size (tokens D), and compute budget C .
- They're a tool for reasoning about *how to spend your training budget efficiently*.
- But it also allows one to see track effective hyperparams (LR, batch size, etc.) changes with N , C , or D .
- **Lesson:**
 - Don't overtune your hyperparameters at small scales and expect to use them at large
 - Instead develop predictive metrics based on parameters:

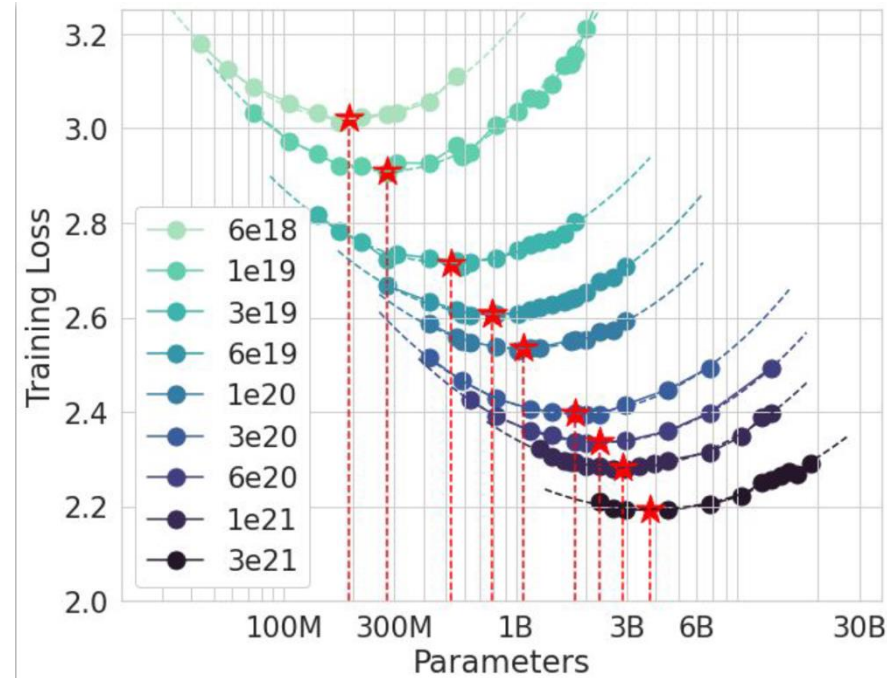
IsoPlots (Isocontours)



Predictive models for parameters

- Overall, IsoPlots show how loss depends on three axes: model size (parameters N), dataset size (tokens D), and compute budget C .
- They're a tool for reasoning about *how to spend your training budget efficiently*.
- But it also allows one to see track effective hyperparams (LR, batch size, etc.) changes with N , C , or D .
- **Lesson:**
 - Don't overtune your hyperparameters at small scales and expect to use them at large
 - Instead develop predictive metrics based on parameters:
- Most of this work has typically been done at industry labs who have the compute to run these experiments.
- Recent open-source efforts from academia/non-profit:
 - OLMo series of models from Ai2.
 - Marin project from Stanford/Together Compute:
 - <https://marin.community/>

IsoPlots (Isocontours)



Pretraining summary

- **IsoPlots:** for a fixed compute, which combination of parameters give you the best bang for the buck.
- For different compute budgets, plot IsoPlots and observe trends. Make predictions about larger budgets before actually training them.
- Very hard to do large scale training in academia but you can still find meaningful trends at smaller scale to predict performance at larger scale even with modest academic compute.

Some related concepts that came after large scale pretraining

- **In-context Learning:** Pretrained models can solve many NLP tasks using demonstrations in the input.
 - Input: $(x_1, y_1), (x_2, y_2), (x_3$
 - Expected model output: y_3 .
- **Chain-of-thought Prompting:** Demonstrations can include “step-by-step” reasoning (r) of how to solve the task.
 - Input: $(x_1, r_1, y_1), (x_2, r_2, y_2), (x_3$
 - Expected model output: r_3 , followed by y_3 .
 - Why: shown to improve performance over just demonstrating with x and y .
 - We will study the modern version of this in detail where you post-train models to “reason”.

In-context Learning (ICL) or Few-Shot Prompting

- LLMs have the ability to “learn” to complete tasks through training examples in the prompt
- The recipe is simple:
 - Take a small number of annotated training examples
 - Convert them using verbalizer templates
 - Concatenate them and follow with the target input
 - The completion will be the label of the input

the movie's acting could've been better, but the visuals and directing were top-notch.



Review: The cinematography was stellar; great movie! Sentiment (positive or negative): positive
Review: The plot was boring and the visuals were subpar.
Sentiment (positive or negative): negative
Review: The movie's acting could've been better, but the visuals and directing were top-notch.
Sentiment (positive or negative):



LLM



positive

$\bar{y}$

In-context Learning (ICL)

Performance

Providing (some) demonstrations almost always leads to improvements

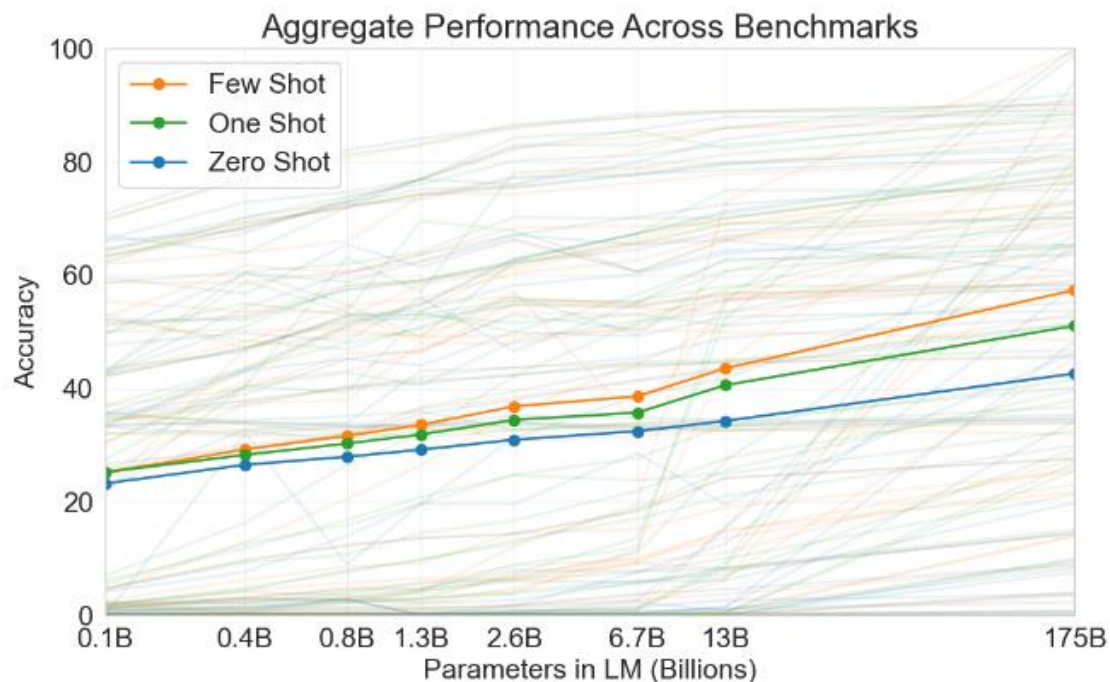


Figure 1.3: Aggregate performance for all 42 accuracy-denominated benchmarks While zero-shot performance improves steadily with model size, few-shot performance increases more rapidly, demonstrating that larger models are more proficient at in-context learning. See Figure 3.8 for a more detailed analysis on SuperGLUE, a standard NLP benchmark suite.

Chain-of-thought (COT) Prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain-of-thought (COT) Prompting

OR just “tell” the model to reason in steps

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. **X**

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. **✓**

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 **X**

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. **✓**

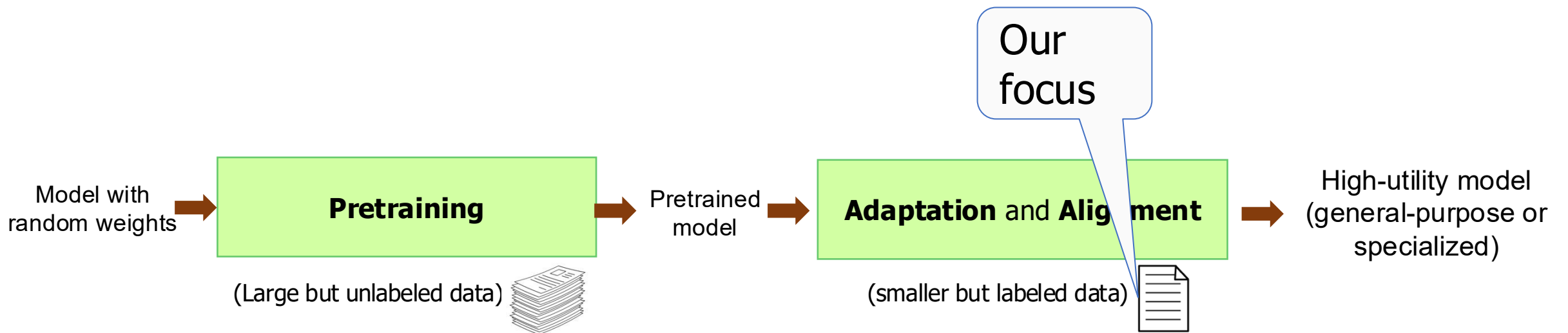
Later: we can train models to think step-by-step or “reason”

Post Training

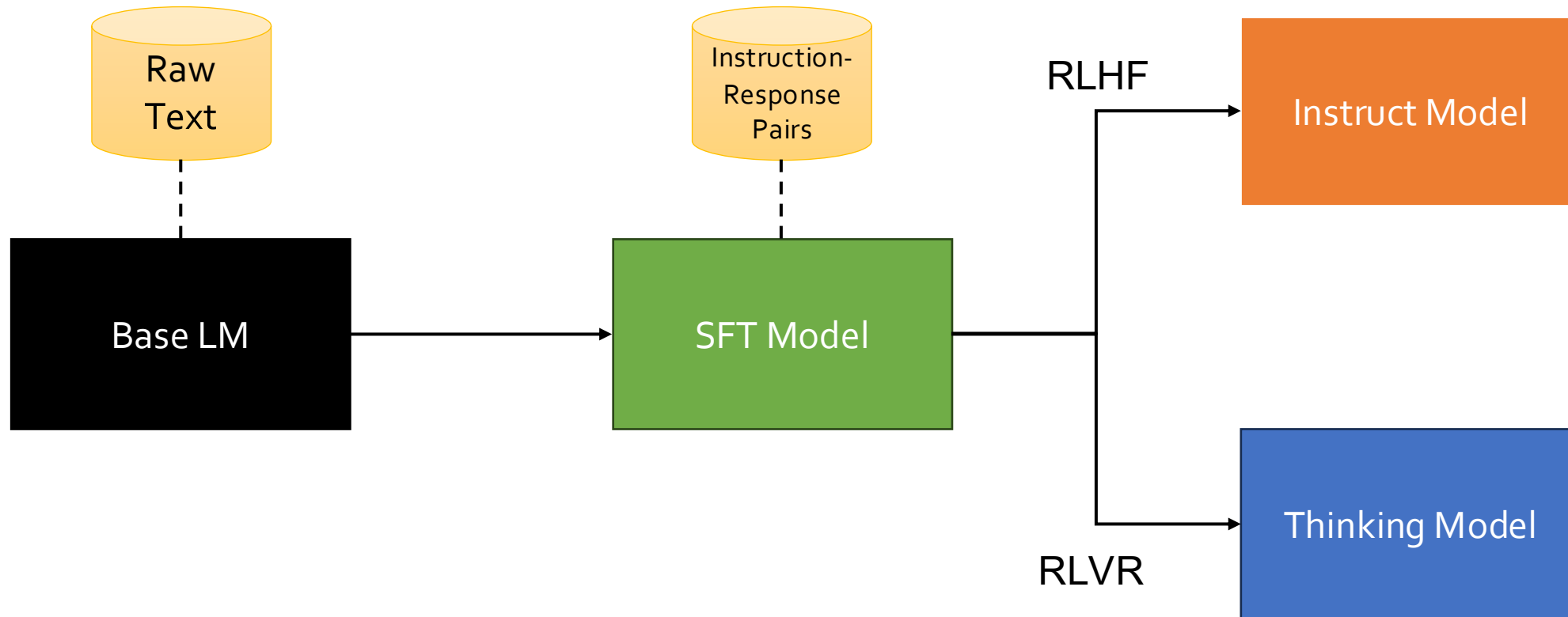
or Alignment

[Mis]Alignment in Language Models

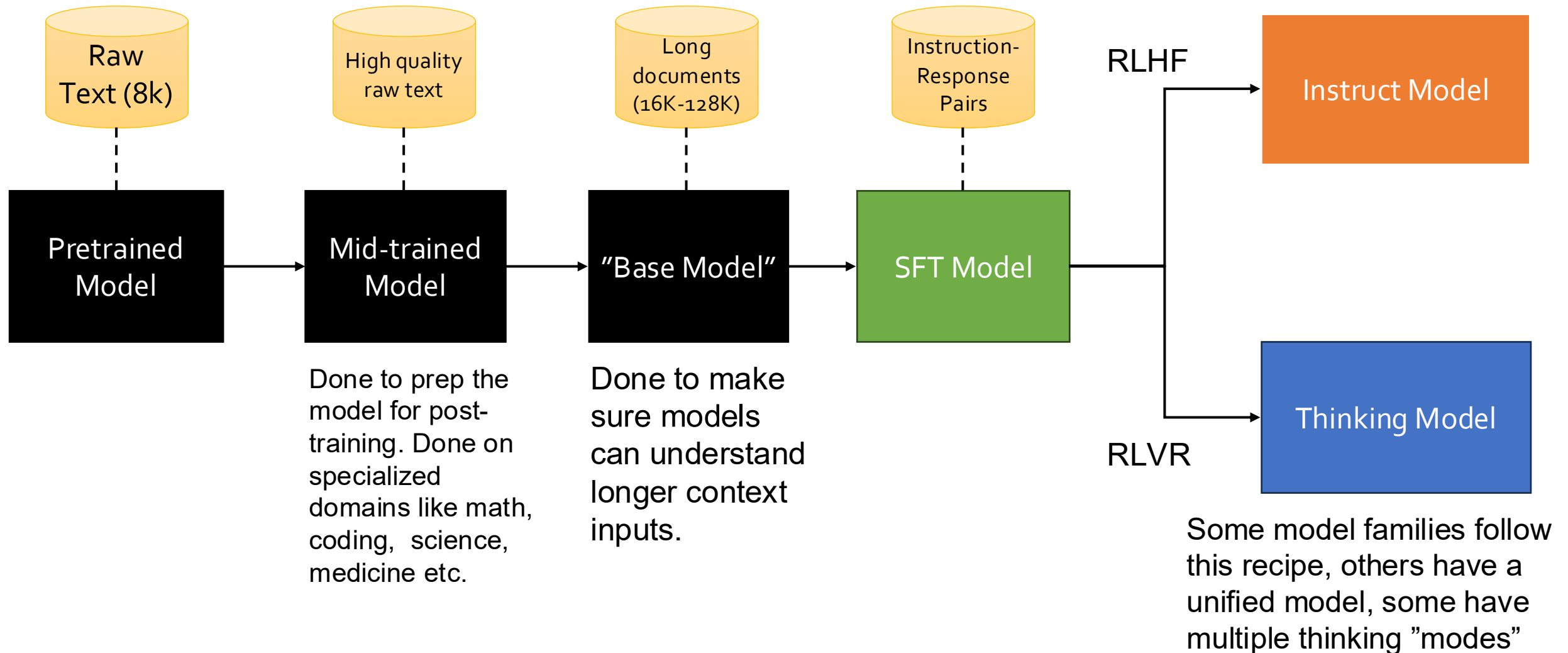
- There is a mismatch between what **pre-trained** models can do and what we want.
- Addressing this gap is the focus of “alignment” research.



Overview of post-training



A more accurate modern LM training pipeline

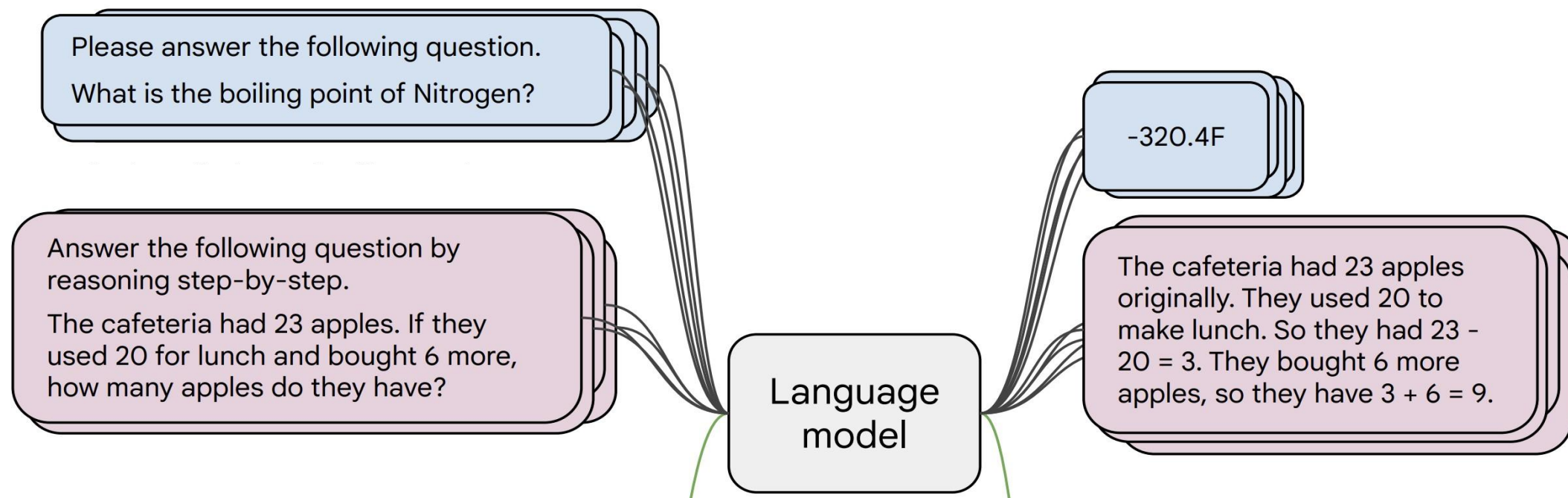


Aligning Language Models: Instruction-tuning

AKA Supervised Finetuning (SFT)

Instruction-tuning

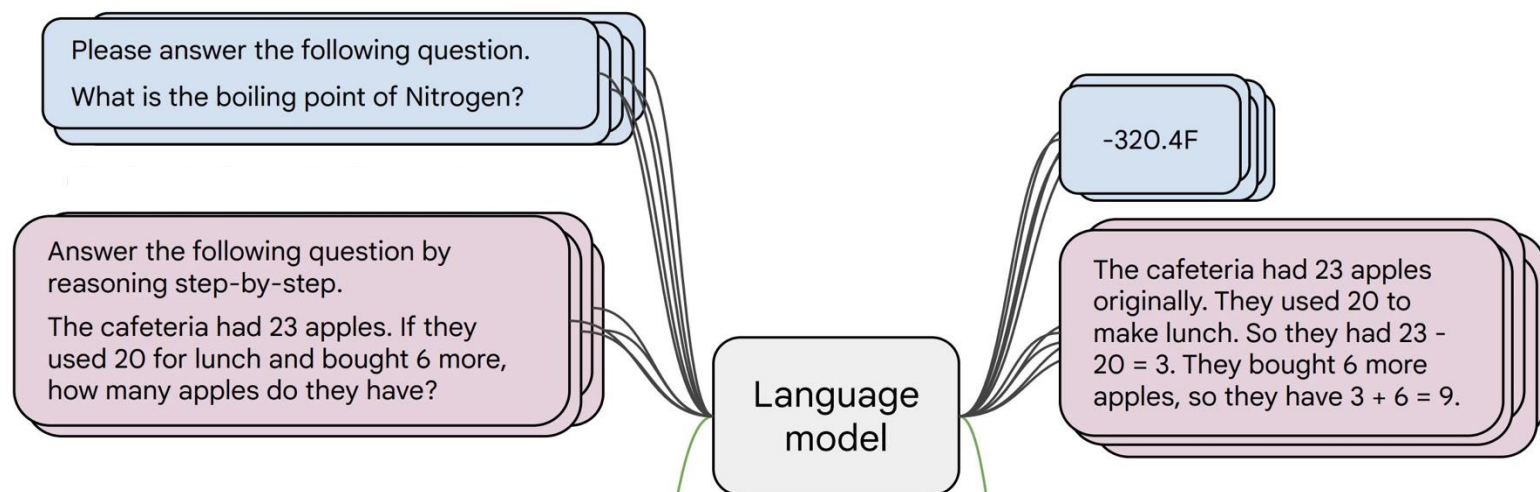
- **Finetuning** pre-trained LMs to map **instructions** to their corresponding **responses**.



Instruction-tuning

[Weller et al. 2020. Mishra et al. 2021; Wang et al. 2022, Sanh et al. 2022; Wei et al., 2022, Chung et al. 2022, many others]

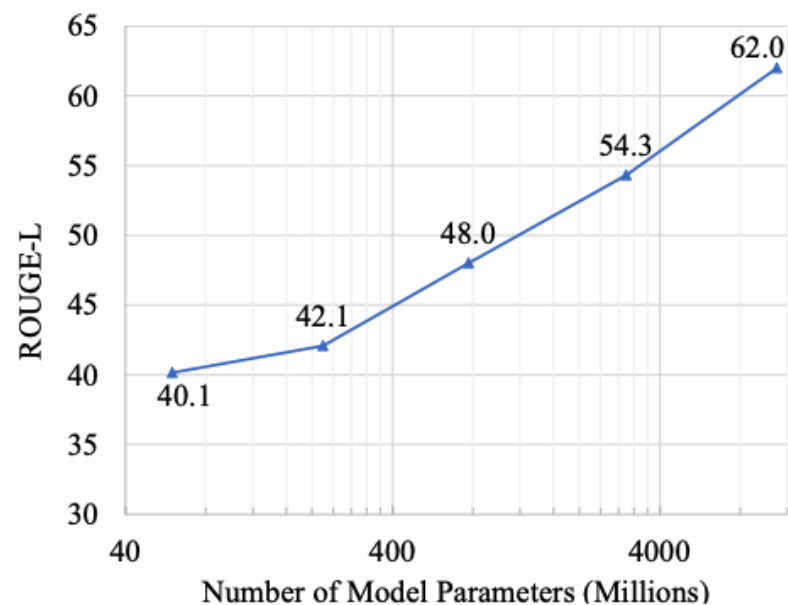
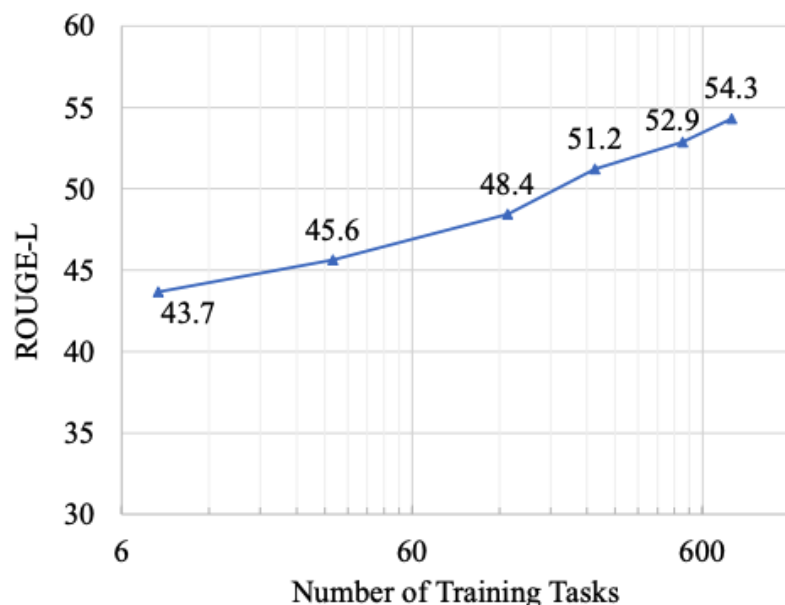
1. Collect examples of (instruction, output) pairs across many tasks and finetune an LM



2. Evaluate on **unseen** tasks

Inference: generalization to unseen tasks

Scaling Instruction-Tuning



Linear growth of model performance
with **exponential** increase in **observed tasks** and **model size**.

[Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks, Wang et al. 2022](#)

Cross-Task Generalization via Natural Language Crowdsourcing Instructions (Mishra et al., 2022)

Where does SFT data come from?

- Early work: convert NLP datasets for various tasks into instructional format. For example,
 - Input: text, output: sentiment label (such as 0 for negative and 1 for positive)
--> what is the sentiment rating of this review: text, output: sentiment label
in textual format (this review is positive).
- Ask humans to write questions and answers.
- Use in-context learning to generate instructions and outputs using a seed-set (see [self-instruct](#)).
 - Along with lots of filtering and other tricks to maintain quality
- Now: typically a mix of all of this (with a large proportion of synthetic examples).

Today's paper

- Title: STaR: Self-Taught Reasoner Bootstrapping Reasoning With Reasoning
- Builds on top of SFT / chain-of-thought prompting.

Stakeholder 

STaR Reasoner: Motivation

- Rationales typically improve task performance – in human decision making and also for language models as shown recently via chain-of-thought prompting
- Setup: Given input x , the model first generates a rational/chain-of-thought/reasoning chain r . Conditioning on x and r , the model then generates the final output y .
 - Compared to directly generating the output y given x , r improves performance.
- Typical ways to generate the chain of thought: use demonstrations/in-context examples or do prompting (ask models to think step-by-step).
 - This can work sometimes but still has a high error rate.
 - Solution: we need to train models to first generate r before y .

Training models to generate reasoning traces

- Handwrite examples or pay annotators to hand write examples
 - Issues?
 - Too expensive, hard to get diversity.
- Use templates as in create algorithmic ways to create r
 - Also hard to get diverse examples
 - Many kinds of reasoning are hard to templatize
 - For many tasks, multiple kinds of reasoning paths are possible to arrive at the right answer, some might be harder to templatize than others.

This paper's solution: create synthetic reasoning traces using the LM itself.

- Start with a dataset containing only input x and output y .
- For each example input x , using few-shot prompting to generate a rationale r and final answers y' . Keep examples (x, r, y) for which the answer is correct.
- Finetune the model to generate reasoning traces and answer.
- Repeat the process (generate more data with finetuned model and refinetune the model)

This loop eventually fails, because the model cannot get all answers correct

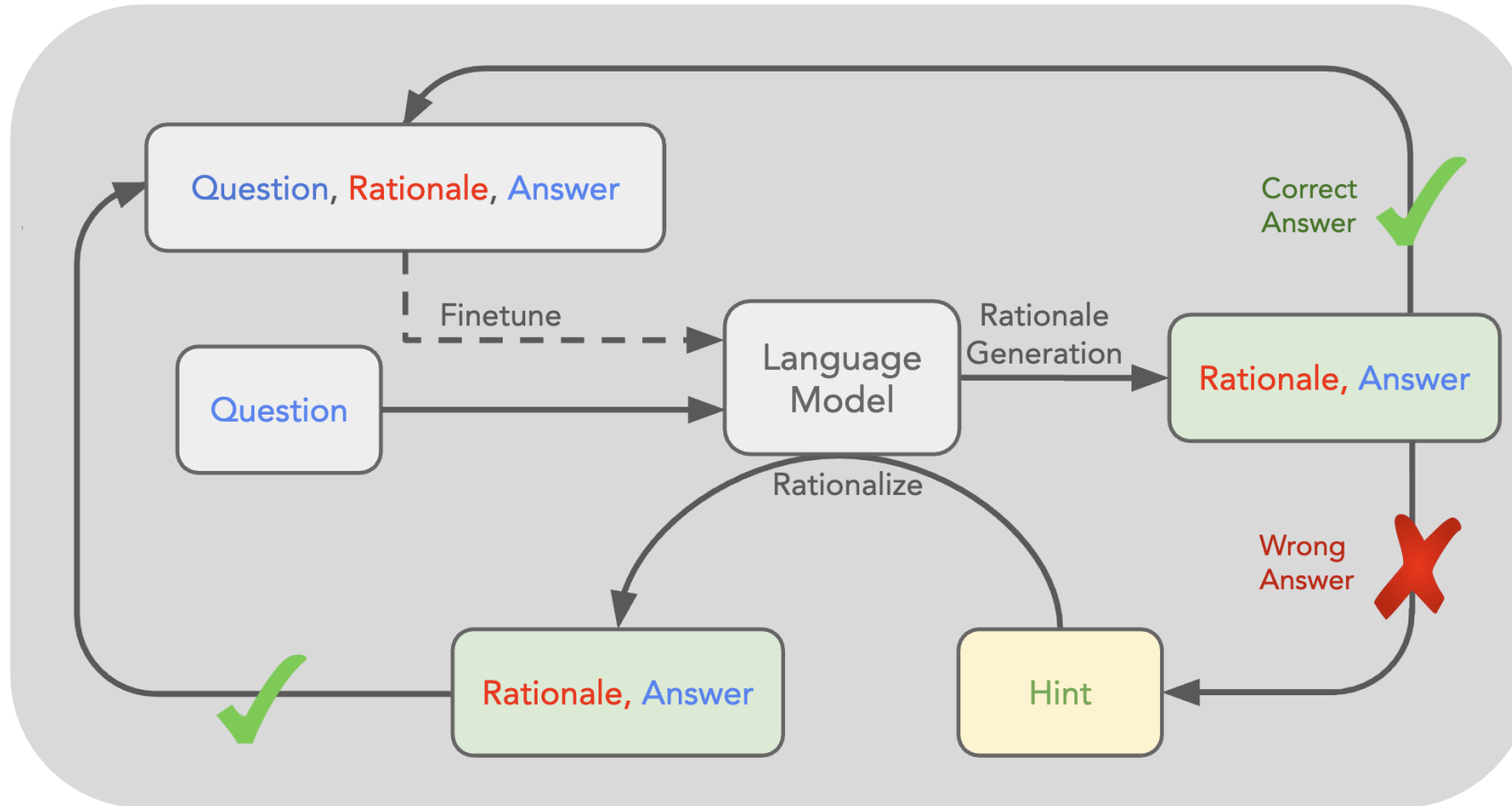
This paper's solution: create synthetic reasoning traces using the LM itself.

- Start with a dataset containing only input x and output y .
- For each example input x , using few-shot prompting to generate a rationale r and final answers y' . Keep examples (x, r, y) for which the answer is correct.

If y' is incorrect: prompt the model with the correct answer to generate a new rationale, include this example with the new rationale in the dataset

- Finetune the model to generate reasoning traces and answer.
- Repeat the process (generate more data from the finetuned model and refinetune the base model)

Self-Taught Reasoner or STaR



Q: What can be used to carry a small dog?
Answer Choices:
(a) swimming pool
(b) basket
(c) dog show
(d) backyard
(e) own home
A: The answer must be something that can be used to carry a small dog. Baskets are designed to hold things. Therefore, the answer is basket (b).

Results – n-digit summation

Input:

6 2 4 + 2 5 9

Target:

<scratch>

6 2 4 + 2 5 9 , C: 0

2 + 5 , 3 C: 1

6 + 2 , 8 3 C: 0

, 8 8 3 C: 0

0 8 8 3

</scratch>

8 8 3

Figure 3: A visualization of a 3-digit arithmetic problem with a scratchpad. C corresponds to the carry from the previous digit's summation.

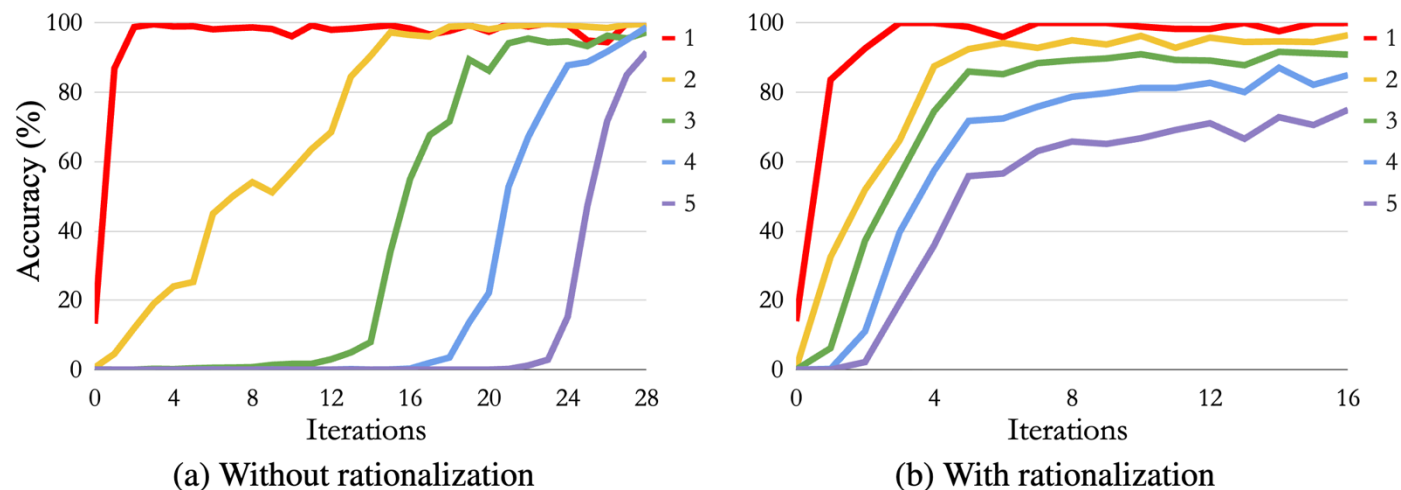


Figure 4: A visualization of the accuracy of n -digit summation with each iteration of STaR with and without rationalization for arithmetic. Each series corresponds to the accuracy of summing two n -digit numbers.

Results – n-digit summation

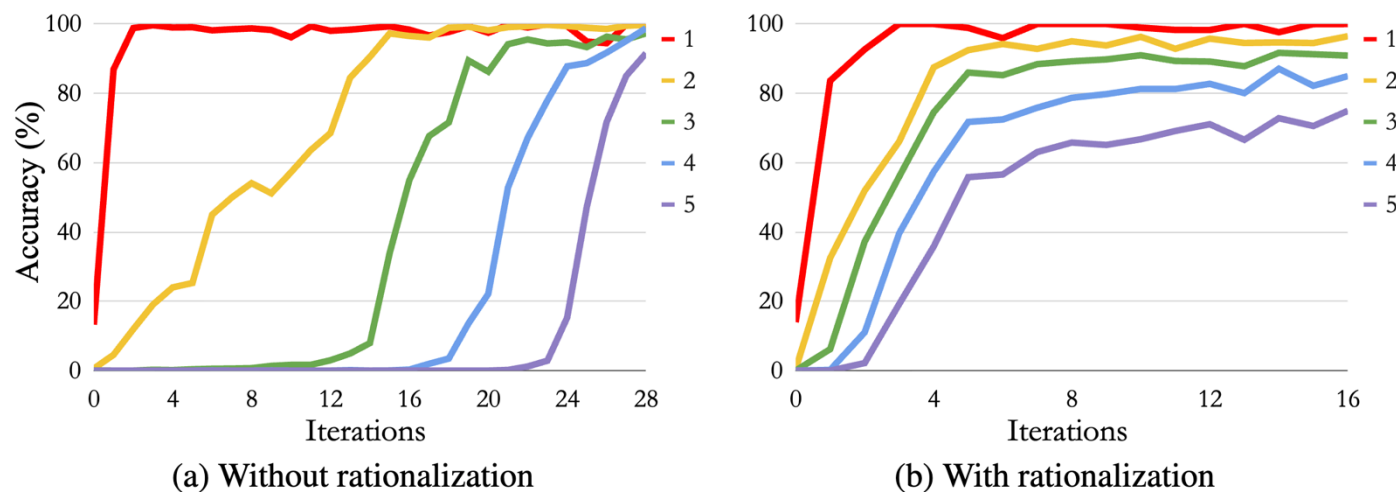


Figure 4: A visualization of the accuracy of n -digit summation with each iteration of STaR with and without rationalization for arithmetic. Each series corresponds to the accuracy of summing two n -digit numbers.

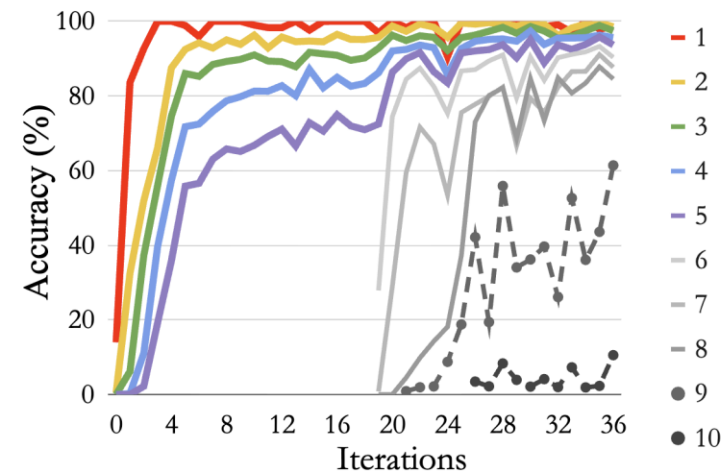


Figure 5: We introduce additional digits to STaR with rationalization at the 20th iteration.

Shows some generalization to higher digits summation not trained on.

Results – Commonsense QA (multiple choice)

Q: John wants to eat at an Indian restaurant in on top of the space needle, where would he have to go?

Answer Choices:

- (a) reservation (b) curry
- (c) british town (d) seattle
- (e) large city

A: The answer must be a place with a space needle. The space needle is a landmark in Seattle. Therefore, the answer is Seattle (d).

Table 1: We evaluate several baselines, including a few-shot GPT-J evaluation both with and without scratchpads, a GPT-J baseline finetuned to directly predict the answer, and STaR with and without rationalization applied to GPT-J. We use CoT to denote non-STaR models outputting rationales, and Direct to indicate those directly predicting the final answer. Note the final STaR model is trained on 78.2% of the training dataset with rationale generation, and an additional 8.5% from rationalization.

	CQA Dev Set Accuracy (%)	Train Data Used (%)
<i>GPT-3 Direct Finetuned [29]</i>	73.0	100
Few-shot Direct GPT-J	20.9	~0
Few-shot CoT GPT-J ³	36.6	~0
Few-shot CoT LaMDA 137B [6]	55.6	~0
GPT-J Direct Finetuned	60.0	100
STaR without rationalization	68.8	69.7
STaR with rationalization	72.5	86.7

Results – Commonsense QA (multiple choice)

Q: John wants to eat at an Indian restaurant in on top of the space needle, where would he have to go?

Answer Choices:

- (a) reservation (b) curry
(c) british town (d) seattle
(e) large city

A: The answer must be a place with a space needle. The space needle is a landmark in Seattle. Therefore, the answer is Seattle (d).

Q: They prided themselves on being a wealth of knowledge, and that's why many chose to attend their what?

Answer Choices:

- (a) book store (b) university (c) meeting
(d) class (e) encyclopedia

~~A: The answer must be a place where people go to learn about things. The answer is university (b).~~

A: The answer must be a place where people go to learn about things. Universities are places where people go to learn about things. Therefore, the answer is university (b).

Table 1: We evaluate several baselines, including a few-shot GPT-J evaluation both with and without scratchpads, a GPT-J baseline finetuned to directly predict the answer, and STaR with and without rationalization applied to GPT-J. We use CoT to denote non-STaR models outputting rationales, and Direct to indicate those directly predicting the final answer. Note the final STaR model is trained on 78.2% of the training dataset with rationale generation, and an additional 8.5% from rationalization.

	CQA Dev Set Accuracy (%)	Train Data Used (%)
<i>GPT-3 Direct Finetuned [29]</i>	73.0	100
Few-shot Direct GPT-J	20.9	~0
Few-shot CoT GPT-J ³	36.6	~0
Few-shot CoT LaMDA 137B [6]	55.6	~0
GPT-J Direct Finetuned	60.0	100
STaR without rationalization	68.8	69.7
STaR with rationalization	72.5	86.7

Results – human evaluation of rationales

- In a human ranking study, STaR rationales were 30% more likely to be preferred over few-shot CoT rationales ($p = .039$).
- STaR rationales were also preferred over the available human-written rationales: crowdworkers were 74% more likely to rank STaR higher ($p < .001$).
- Important caveat: this does not imply human-level reasoning quality; the authors argue it instead highlights how hard it is to elicit consistently good human rationales.

Other misc details

- Failure modes – the authors provide many examples where the rationale is false but answers are correct or vice-versa
 - Important discussion point for many future classes (chain-of-thought faithfulness etc).
- The models must be capable enough to bootstrap reasoning (that is few-shot performance must be good enough).
 - Related to current hype around recursive self-improvement.
- Including few-shot examples during training helps boosts results even more sometimes.
 - Training models to learn from demonstrations, still a big part of most training pipelines.
 - This contrasts with ICL from pretrained models where this ability is to have said to “emerge” with scale - multiple studies disagree however.

Reviewer guidelines

1. Brief Summary
2. Reviewer Comments
3. Conclusion and Discussion

The aim of peer review is to provide authors with constructive feedback from subject experts, so that they can make improvements to their manuscript (Caveat: the paper is already accepted to a major conference but we will follow this principle in spirit).

Brief Summary of STaR Reasoner

- A method for "bootstrapping reasoning"
 - starting with a few examples of reasoning in a domain and producing supervised explanations for that domain.
 - This uses a combination of two methods: generating an explanation and an answer and keeping the explanation if the answer is right, plus post-hoc rationalizing correct answers for cases where the model makes the wrong prediction.
 - That is, the correct answer is used to "prime" the language model to generate an explanation geared towards that answer.
- Experiments are conducted on an arithmetic dataset, on GSM8k, and on CommonsenseQA. Rationalization leads to faster learning for arithmetic, though not necessarily better "final" performance. On CommonsenseQA, the rationalized model does perform better and is able to use more of the training data.
- Human annotation shows that these generated explanations are of good quality.

Strengths

- Well motivated and naturally builds on recent work on chain-of-thought prompting and self-training/synthetic data generation.
- Tackles diversity of tasks – arithmetic, mathematical, commonsenseQA where different kind of reasoning abilities are required.
- Experiments are rigorous, different modeling decisions such as rationalization are properly ablated.

Weaknesses

- Main experiments are with a synthetic task – digit summation, other experiments like commonsenseQA are multi-choice, where wrong reasoning + right answer has a high chance of happening.
- Rationales generated by the model may be incorrect even though it generates the right answer.
 - Can get worse if the model feeds these possibly flawed explanations to itself.

Archeaology: STaR Reasoner

Major ideas in this paper

- Rationales and explanations to make predictions / explain predictions.
- Training on model predictions
- Using additional information to generate model predictions (few-shot examples) which is then discarded during training

Reasoning, Rationales, Explanations...

- Research in making language models/NLP models explain their decisions (i.e. rationalize their predictions) or using explanations in natural language to make decisions has been going on since ~2016 – some of the earliest papers in this space: [Towards Robust Interpretability with Self-Explaining Neural Networks](https://arxiv.org/abs/1603.08507), <https://arxiv.org/abs/1603.08507>, <https://aclanthology.org/D16-1011/>
 - Research in making “explainable models” in general has been going on for decades.
- This research falls in the general area of interpretability – trying to understand why and how models made certain predictions.
 - Based on what data they were trained on
 - Based on the inner workings in the model’s computation graph
 - Based on the outputs they generated before they generated the final prediction
 - ...
- Many of these papers use terminology which is different from the currently popular terminology – reasoning.



Self-training, Rejection Finetuning

- General idea of self-training (a recent survey: <https://arxiv.org/html/2202.12040v6>)
 - Use a trained model to make predictions on unlabeled data
 - Keep some of them (often based on confidence)
 - Retrain model on the filtered data and obtain improvement.
- Rejection Finetuning (the term was coined after this paper but the ideas have existed).
 - For a given input, generate outputs from a model, keep them if they satisfy a criterion (typically correctness in math/logic problems or another scoring function like a reward model – discussed in a few slides).
 - SFT the model on this data.
 - Repeat
 - Related paper: <https://arxiv.org/abs/2304.06767>, <https://arxiv.org/abs/2308.01825>



Context Distillation

- General idea: provide additional context, such as demonstrations, to the LM prompt to generate desirable outputs and then training a (potentially different) model to generate desirable outputs *without the additional context* is now very common in LM development.



Reasoning models as we know them today

- OpenAI o1 was one of the first “thinking” models, followed shortly by deepseek r1.
- Both models conduct reinforcement learning to further improve reasoning performance.
 - Some rumors suggest that o1 used recipes similar to the STaR paper (and other variants published after).

Visionary: STaR Reasoner



Exploration: How to make this work when the correct answer is not available

- This paper assumes access to the correct answer.
 - If predicted answer is correct, include rationale and label in the training set,
 - Otherwise generate rationales.
- How to make this work for “unverifiable tasks” or tasks with no one correct answer?



AI Reviewer (Offline)

- <https://paperreview.ai/review>
- Access code: I-liSkqgcneLXKJ-
TOgmML3UEAnQK2NX4IMAmnIba4ME

Reproducer: Can the results be reproduced?

- **Mostly, but not perfectly.** The authors release an official code repository: fine-tuning code, setup instructions, and utilities for working with the GPT-J checkpoints used in the experiments. The paper also uses publicly available datasets, so the core method is reproducible in principle [include links to data/code/other artifacts]
- Reimplementation is quite environment-specific: it was built around JAX/TPUs, requires old pinned JAX versions for the GPT-J setup, and the repository notes that some newer model code does not have publicly released weights.
- Reproducibility verdict: Mostly reproducible
Code: public
Data: public
Model/checkpoint access: non-trivial to use (as of 2026)
Hyperparameters/method details: yes
Environment/setup: somewhat brittle/dated
Can I easily reproduce the headline result today? possibly, but not exactly

Theme Context (Offline)

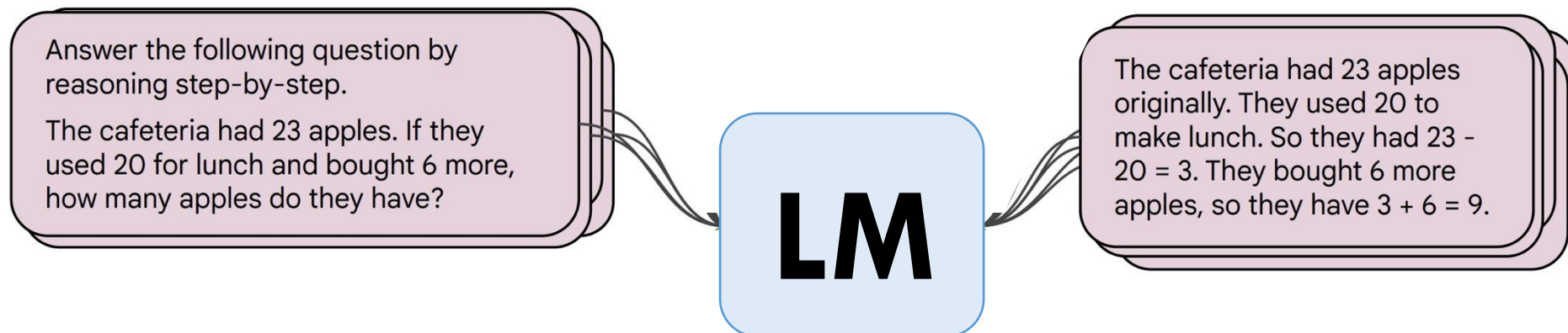
- STaR asks whether a model can bootstrap better reasoning from its own generated solutions rather than relying entirely on human written rationales. It marks an important transition from *eliciting* reasoning through chain-of-thought prompting to *training models on reasoning they generate themselves*. It is a precursor to today's RLVR-based reasoning models.
- Constraint: getting human written reasoning traces is expensive.

Questions?

Back to foundations – limitations of SFT

Limits of SFT

1. Difficult to collect diverse data – could be mitigated by bootstrapping from models themselves (like STaR)
2. Resulting models may not be good at open-ended generation tasks.
 - Incentivizes word-by-word rote learning => The resulting LM's **generality/creativity** is bounded by that of **their supervision data**.



Limits of SFT

1. Difficult to collect diverse data.
2. Resulting models may not be good at open-ended generation tasks.
 - Incentivizes word-by-word rote learning => The resulting LM's **generality/creativity** is bounded by that of **their supervision data**.
3. Resulting models may hallucinate more regularly.
 - Labeled data is collected agnostic to the LM's knowledge => there might be a mismatch between labeled data and LM knowledge.

Aligning Language Models: Reinforcement Learning

Reinforcement Learning: Intuition

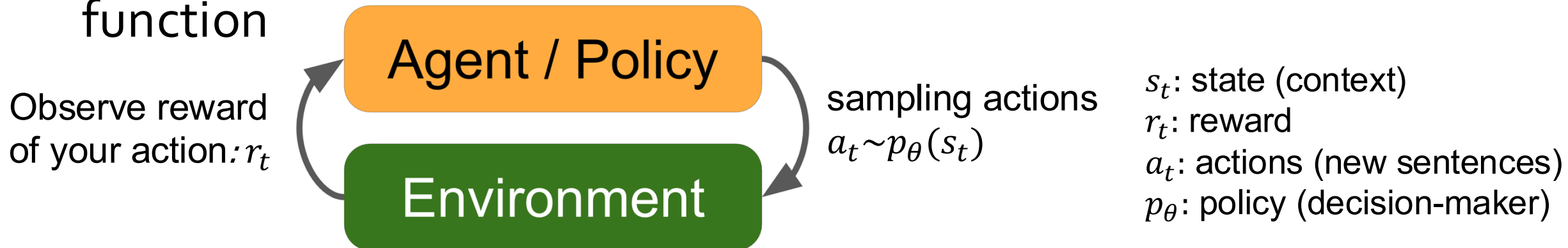
Action here: generating responses/token



Reward: a scalar value serving as a proxy for if the agent does what it's supposed to.

Reinforcement Learning: Formalism

- An agent **interacts** with an environment by taking **actions**
- The environment returns a **reward** for the **action** and a **new state** (representation of the world at that moment).
- Agent uses a **policy function** to choose an action at a given **state**.
- We need to figure out: (1) reward function and (2) the policy function



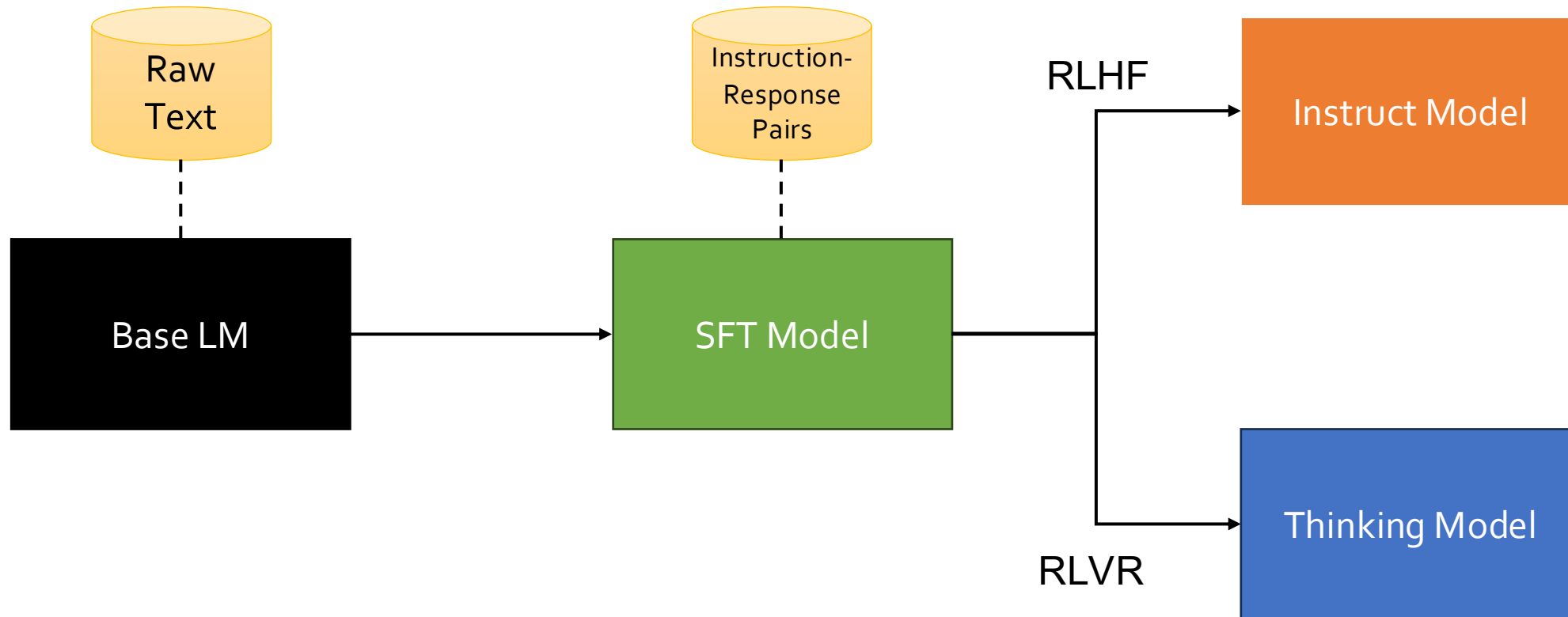
Reinforcement Learning in LMs

Two major paradigms based on which rewards are used.

1. Reinforcement Learning from Human Feedback (human preferences are converted to rewards – inherently subjective) – RLHF
2. Reinforcement Learning from Verifiable Rewards (such as answer correctness, similar to the STaR paper) -- RLVR

RLHF was the major paradigm starting 2021-2024, starting 2024/2025, most major labs have a thinking model which is trained with RLVR.

(Crude) overview of post-training



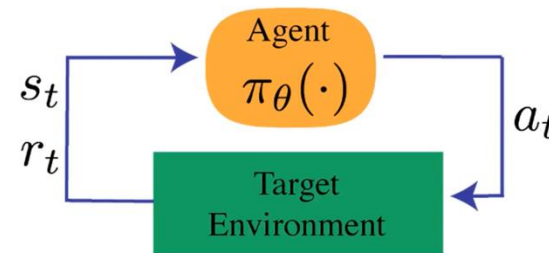
RLVR

RLVR Setup



- Policy Model: A language model, can either just be pretrained or SFTed (typically)
 - Given a prompt, this model is expected to first generate a reasoning trace followed by the final response.
 - `X <think>reasoning trace....</think><response>actual response</response>`
- The model receives a reward based on the correctness on the content in the `<response></response>` tags.
- The RL process updates the model to maximize the expected reward.

Notation



- a_t : action (next token(s) generated)
- s_t : state (text in the model's context / prompt)
- $\pi_{\theta}(a|s)$: policy function, parameterized by θ ; distribution over actions at state s (our language model which we want to optimize)
- r_t : reward associated with a given action and state.

There are other notations we will gradually introduced like value model, baselines etc, and some we will skip for simplicity such as Return, Q-function. Please check [RLHF textbook](#) for details.

Reinforcement Learning from Verifiable Rewards

- Imagine a reward function: $R(a; \text{prompt}) \in \mathbb{R}$ for any output a to a prompt.

- Good generation is equivalent to finding reward-maximizing outputs:

$$\mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

- What we need to do:

- Find the best generative model p_{θ} that maximizes the expected reward:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$



Step 1: Reward function R

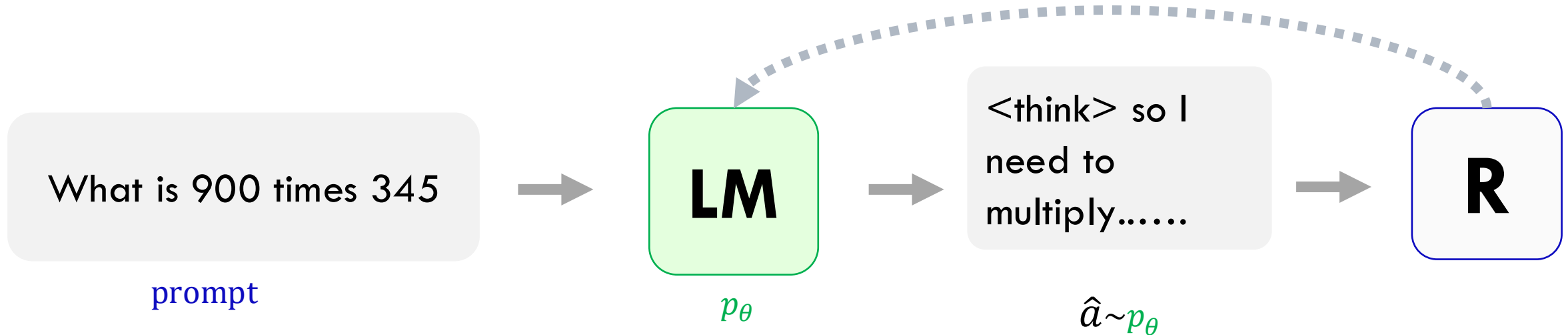
- In RLVR, the most basic reward is typically a binary value (other variants also exist):
 - 1 if the model answer is correct.
 - 0 if the model answer is wrong.
- How to measure correctness?
 - Today we will only talk about tasks whose correctness is verifiable – such as math reasoning (by simply matching the output with the correct answer) or code generation (where compilation errors / test cases can verify correctness of code).
 - Future: approximate correctness with rubric-based rewards / other means.

Step 2: Optimizing the Policy Function



- Policy function := The model that makes decisions (here, generates responses)
- How do we change our LM parameters θ to maximize this?

$$\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$



Step 2: Optimizing the Policy Function



- Policy function := The model that makes decisions (here, generates responses)
- How do we change our LM parameters θ to maximize this?

$$\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

How do we estimate the gradient of this expectation?

Notice that R is not directly dependent on θ . (You can't compute its grad with respect to θ)

- Turns out that we can write this "gradient of expectation" to a simpler form.

Policy Gradient [Williams, 1992]



- How do we change our LM parameters θ to maximize this?

$$\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} \leftarrow \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}_{\hat{a} \sim p_{\theta}} [R(\hat{a}; \text{prompt})]$$

- With a bit of math, this can be approximated as Monte Carlo samples from $p_{\theta}(a)$:

$$\nabla_{\theta} \mathbb{E}_{a \sim p_{\theta}} [R(a; \text{prompt})] \approx \frac{1}{n} \sum_{i=1}^n R(a_i; \text{prompt}) \nabla_{\theta} \log p_{\theta}(a_i; \text{prompt})$$

- This is “**policy gradient**”, an approach for estimating and optimizing this objective.
- Oversimplified. For full treatment, see [RL textbook](#).

Policy Gradient [Williams, 1992]



Note, $R(s; \text{prompt})$ could be any arbitrary, non-differentiable reward function that we design.

- This gives us the following update rule:

$$\theta_{t+1} \leftarrow \theta_t + \alpha \frac{1}{|\text{samples}| \times |\text{prompts}|} \sum_{p \in \text{prompts}} \sum_{a_i \sim p_\theta(p)} R(a_i; p) \nabla_\theta \log p_\theta(a_i; p)$$

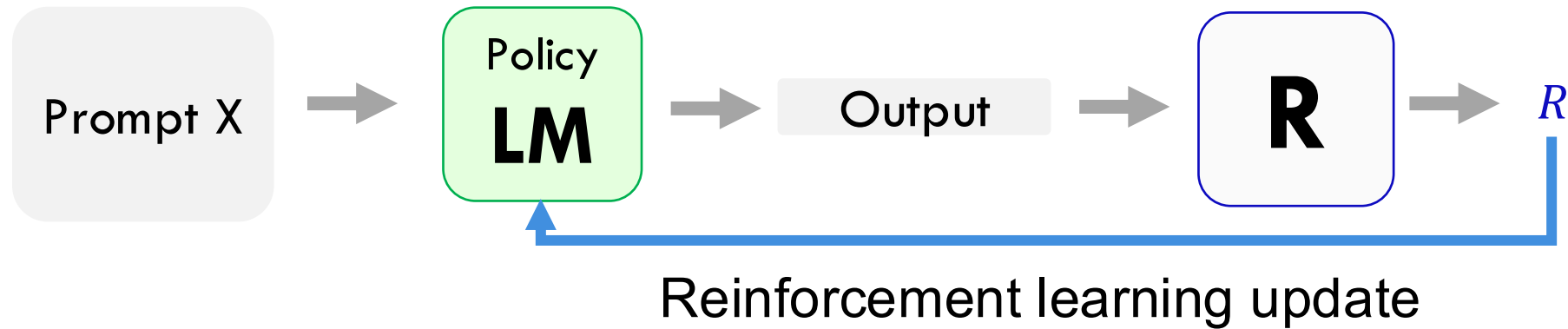
- If $R(a; p)$ is **large**, we take proportionately **large** steps to maximize $p_\theta(a)$
- If $R(a; p)$ is **small**, we take proportionately **small** steps to maximize $p_\theta(a)$

This is why it's called "reinforcement learning":
we reinforce good actions, increasing the chance they happen again.

Summary of basic RL with REINFORCE



- We want to learn a policy (a Language Model) that optimizes against the reward function

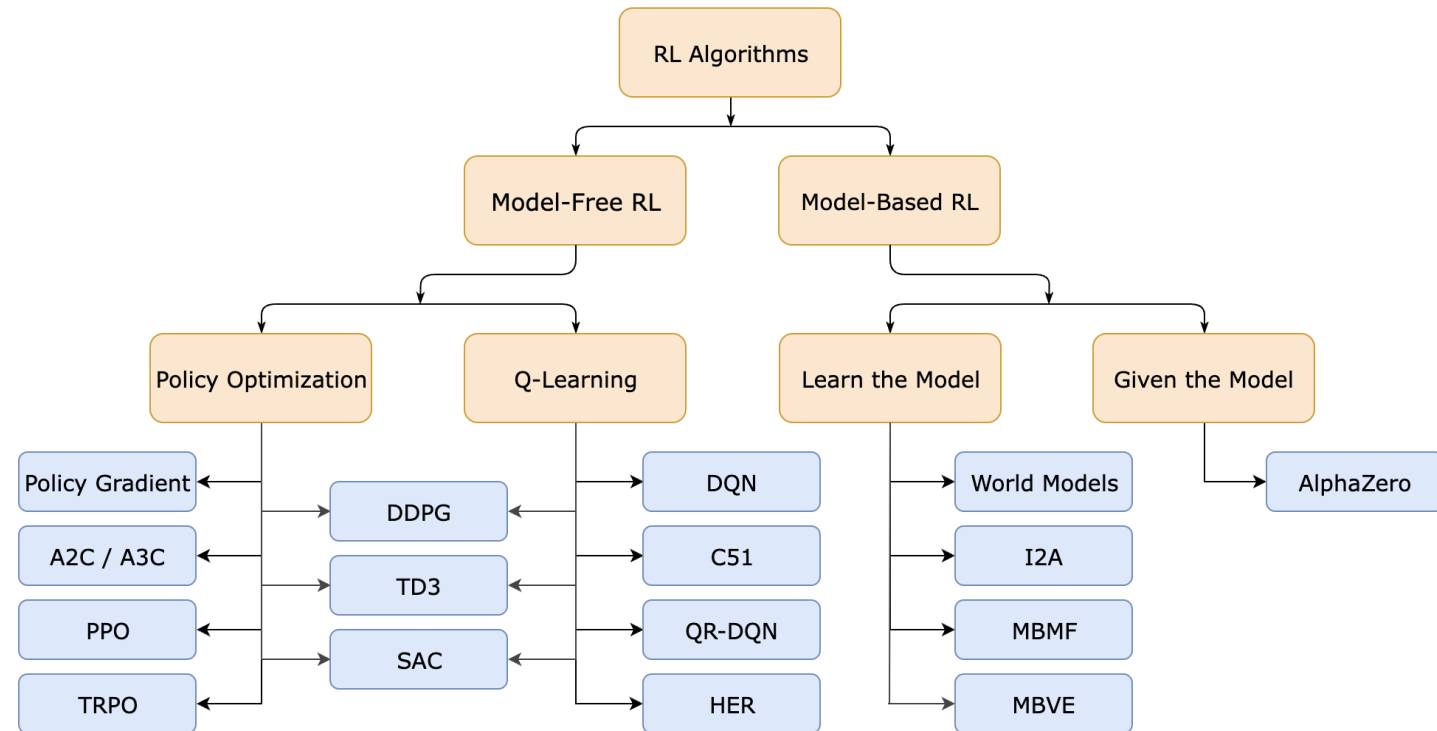


$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^n R(a_i; \mathbf{p}) \nabla_{\theta} \log p_{\theta}(a_i; \mathbf{p})$$

REINFORCE is too simple to work,
what do people *actually* use?

The Bigger Picture

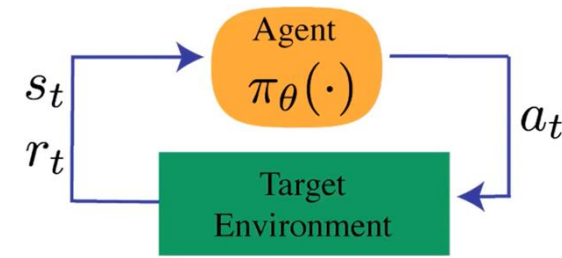
- What we saw was a simple policy gradient algorithm for RL.



Check out Andrew Perrault's courses:
<https://aperrault.github.io/teaching/>

https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html

REINFORCE: Challenges



- **Distribution drift:** While the gradient updates maximize the rewards, it may deviate from natural distribution (it may hack its way to high reward).
 - Generate unnatural looking text, this is typically handled by adding a KL divergence loss to the objective, which minimizes too much deviation from the original policy also known as “reference policy”.
- **High variance:** The gradient estimates suffer from high variance.
 - This may lead to destructively large updates and sample inefficiency.

Modification 1: The baseline estimate

- To reduce the variance of the gradient, we subtract a **baseline estimate** $b_t(s_t)$:

$$g^{\text{VR}} = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) (r_t - b_t)]$$

- Note, by design, the baseline **depends on states s_t but not the action a_t** .
- Therefore, $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) (r_t - b_t)$ is an “unbiased estimator” of $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) r_t$ (Why?)

What can be a baseline?

$$g^{\text{VR}} = \mathbb{E}_{a \sim \pi} [\nabla_{\theta} \log \pi_{\theta} (a_t | s_t) (r_t - b_t)]$$

- A “good” baseline is a function that:
 - doesn’t depend on actions.
 - Minimizes variance:
 - $b^*(s_t) = \operatorname{argmin}_b \mathbb{E}_{a \sim \pi} [|g^{\text{VR}} - E[g^{\text{VR}}]|^2]$
 - $b^*(s_t) = V^{\pi}(s_t) = \mathbb{E}_{a \sim \pi} [r_t(s_t, a_t)]$, also known as the **value function**.
- Full derivation in <https://rlhfbook.com/book.pdf#page=57.27>

Value Function as a Baseline

$$g^{\text{VR}} = \mathbb{E}_{a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (r_t - b_t)]$$

- A “good” baseline is a function that:
 - doesn’t depend on actions.
 - Minimizes variance – value function.
- *In practice, value function is learned since it cannot be directly estimated.*
 - $V_{\omega}^{\pi}(s)$: value of state s , parameterized by ω ; expected reward from here on under policy π , assuming that we’re at state s .

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi} [r_t | S_t = s]$$

$$g^{\text{VR}} = \mathbb{E}_{a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (r_t - V^{\pi}(s))]$$

Crudely this difference
is referred to as
“advantage function”

Policy Gradient with Advantage Function

- Advantage-based Policy Gradient updates:

$$g^{\text{APG}} = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t]$$

- We don't (always) need to compute the absolute benefit of an action, but only how much better it is relative to others (i.e., the **relative advantage** of that action.)
- The advantage function $A^{\pi}(s, a)$ of a policy π quantifies **how much better it is to take a specific action a in state s , over a randomly selecting an action according to $\pi(\cdot | s)$, assuming you act according to π forever after.**
- Now we need an algorithm that updates the policy while estimating the advantages.

Summary so far

- Attempt 1: Policy gradient (distribution drift, variances are too high)

$$\max_{\theta} \mathbb{E}[\log \pi_{\theta}(a_t|s_t) R_t]$$

- Attempt 2: distribution drift – add a KL divergence in the loss to prevent the policy to drift too far away from reference.
- Attempt 3: Policy gradient with advantage function to reduce variance.

$$\max_{\theta} \mathbb{E}[\log \pi_{\theta}(a_t|s_t) A_t]$$
$$A_t(s, a) = r(s, a) - V^{\pi}(s)$$

Other bag of tricks that constitute “PPO”

- PPO (for a long time) was the standard algorithm for RL in LMs.
 - Prevent distribution drift: using the KL divergence term
 -
 - Use advantage formulation we just discussed
 - Value model is learned along with the policy. We haven’t discussed how, but its pretty straightforward.
 - [New] At each iteration, a bunch of samples are generated, also known as rollouts. This is expensive to generate once and discard. One could do multiple model updates with one batch of rollouts.
 - This makes the updates “off-policy”. The expectation of rewards is not correct anymore.
 - Uses “importance sampling ratio” to get the correct estimate for gradients. Read RL textbook for details.
 - [New] Trust region:
 - Do not move too fast to new weights (stay within the trust region). Make small updates by clipping the importance sampling ratio.
- PPO requires three models in memory: original model (reference), policy model (that you are learning), and a value model (that you also learn).

PPO Objective

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]$$

$$\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon & \text{if } r_t(\theta) < 1 - \epsilon \\ 1 + \epsilon & \text{if } r_t(\theta) > 1 + \epsilon \\ r_t(\theta) & \text{else} \end{cases}$$

Recent Extension:

Group Relative Policy Optimization (GRPO)

- PPO has 3 LMs in the mix: value, policy and reference policy and also a fourth one (reward model in case of learned rewards with RLHF, more on this later).
 - Massive memory footprint.
- **GRPO** drops the value model. → Significant reduction of memory usage.
- We had value function in PPO is to estimate “advantage” values.
 - If we find alternative way of estimating advantage, we can drop value function.

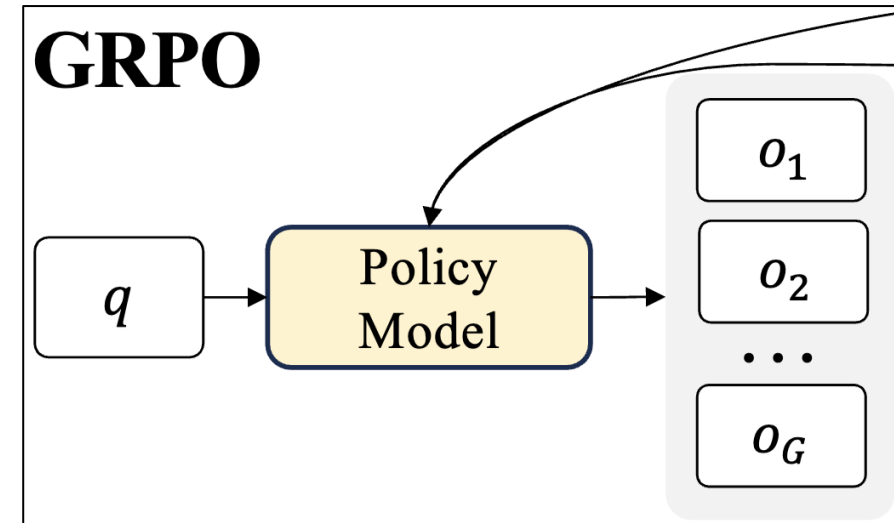
$$\max_{\theta} \mathbb{E}[\log \pi_{\theta}(a_t | s_t) A_t]$$
$$A_t(s, a) = r(s, a) - V^{\pi}(s)$$

GRPO: Key Idea

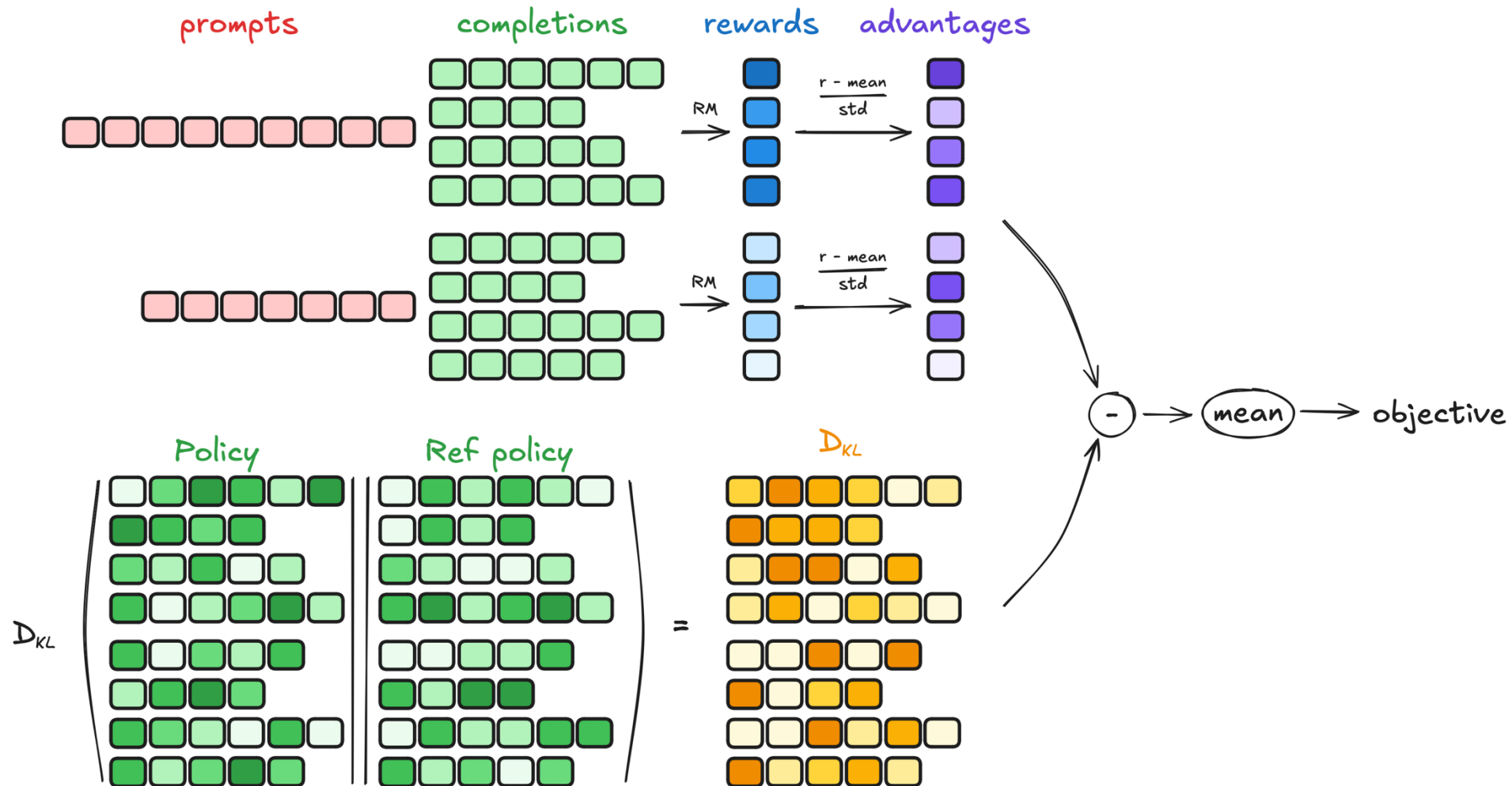
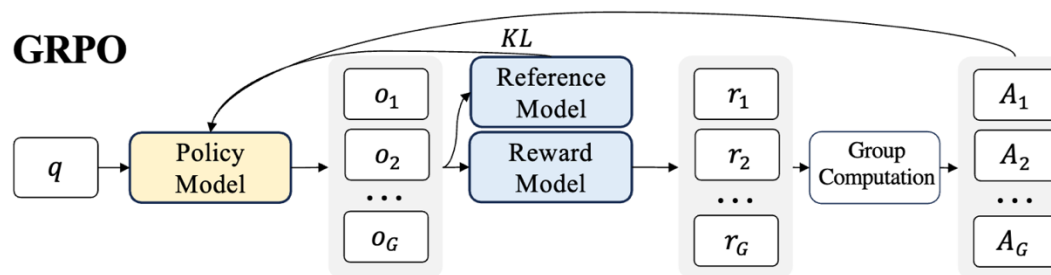
- Execute multiple rollouts from each.
- Given these rollouts, we can estimate the “advantage” function based on **the relative goodness of these responses**.

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

- Advantage of each rollout is simply the gap between its reward compared to the mean reward of other responses, normalized with std.



GRPO



GRPO with rule-based rewards (RL with Verifiable Feedback: RLVR)

2.2.2. Reward Modeling

The reward is the source of the training signal, which decides the optimization direction of RL. To train DeepSeek-R1-Zero, we adopt a rule-based reward system that mainly consists of two types of rewards:

- **Accuracy rewards:** The accuracy reward model evaluates whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for LeetCode problems, a compiler can be used to generate feedback based on predefined test cases.
- **Format rewards:** In addition to the accuracy reward model, we employ a format reward model that enforces the model to put its thinking process between '<think>' and '</think>' tags.

We do not apply the outcome or process neural reward model in developing DeepSeek-R1-Zero, because we find that the neural reward model may suffer from reward hacking in the large-scale reinforcement learning process, and retraining the reward model needs additional training resources and it complicates the whole training pipeline.

```
# Reward functions
def correctness_reward_func(prompts, completions, answer, **kwargs) -> list[float]:
    responses = [completion[0]['content'] for completion in completions]
    q = prompts[0][-1]['content']
    extracted_responses = [extract_xml_answer(r) for r in responses]
    print('-'*20, f"Question:\n{q}", f"\nAnswer:\n{answer[0]}", f"\nResponse:\n{responses}")
    return [2.0 if r == a else 0.0 for r, a in zip(extracted_responses, answer)]

def int_reward_func(completions, **kwargs) -> list[float]:
    responses = [completion[0]['content'] for completion in completions]
    extracted_responses = [extract_xml_answer(r) for r in responses]
    return [0.5 if r.isdigit() else 0.0 for r in extracted_responses]

def strict_format_reward_func(completions, **kwargs) -> list[float]:
    """Reward function that checks if the completion has a specific format."""
    pattern = r"^<reasoning>\n.*?\n</reasoning>\n<answer>\n.*?\n</answer>\n$"
    responses = [completion[0]['content'] for completion in completions]
    matches = [re.match(pattern, r, flags=re.DOTALL) for r in responses]
    return [0.5 if match else 0.0 for match in matches]
```

https://gist.github.com/willccbb/4676755236bb08cab5f4e54a0475d6fb#file-grpo_demo-py-L64-L88

Summary

- RL has many model variants.
- Thus far we have seen:
 - Policy Gradient
 - PPO (barely) – important to remember the idea of value models.
 - GRPO (variants of this will be discussed in future lectures)
- See implementation of alignment algorithms:
<https://huggingface.co/docs/trl/index>

Reinforcement Learning with *Human Feedback*

Reinforcement Learning from Human Feedback



- Imagine a reward function: $R(s; \text{prompt}) \in \mathbb{R}$ for any output s to a prompt.
- The reward is higher when humans prefer the output.
- Good generation is equivalent to finding reward-maximizing outputs:

Expected reward over the course of sampling from our policy (generative model)

$$\mathbb{E}_{\hat{s} \sim p_{\theta}} [R(\hat{s}; \text{prompt})]$$

$p_{\theta}(s)$ is a pre-trained model with params θ we would like to optimize (policy function)

- On the notation:
 - “ $\mathbb{E}$ ” here is an empirical expectation (i.e., average).
 - “ $\sim$ ” indicates sampling from a given distribution.

Reinforcement Learning from Human Feedback

- Imagine a reward function: $R(s; \text{prompt}) \in \mathbb{R}$ for any output s to a prompt.
- The reward is higher when humans prefer the output.
- Good generation is equivalent to finding reward-maximizing outputs:

$$\mathbb{E}_{\hat{s} \sim p_{\theta}} [R(\hat{s}; \text{prompt})]$$



- What we need to do:
 - (1) Estimate the reward function $R(s; \text{prompt})$.
 - (2) Find the best generative model p_{θ} that maximizes the expected reward: $\hat{\theta} = \operatorname{argmax}_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}} [R(\hat{s}; \text{prompt})]$

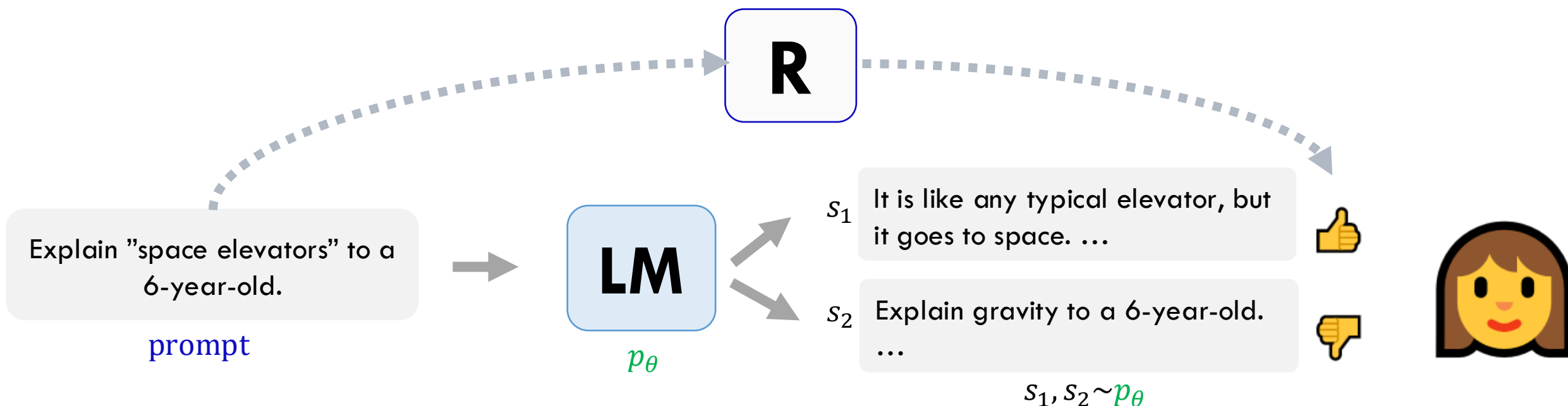
Step 1: Estimating the Reward R

s_1

$$J(\phi) = -\mathbb{E}_{(s^+, s^-)} [\log \sigma(R(s^+; \text{prompt}) - R(s^-; \text{prompt}))]$$

“winning”
sample

“losing”
sample



Step 1: Estimating the Reward R

S_1

$$J(\phi) = -\mathbb{E}_{(s^+, s^-)} [\log \sigma(R(s^+; \text{prompt}) - R(s^-; \text{prompt}))]$$

"winning"
sample

"losing"
sample

R

The reward model returns a scalar reward which should numerically represent the human preference.

Explain "space
elevators" to a 6-year-
old.
prompt

LM

p_θ

s_1

It is like any typical
elevator, but it goes to
space. ...

s_2

Explain gravity to a 6-
year-old. ...

$s_1, s_2 \sim p_\theta$

$R(s_2; \text{prompt}) = 1.2$

$R(s_1; \text{prompt}) = 0.8$

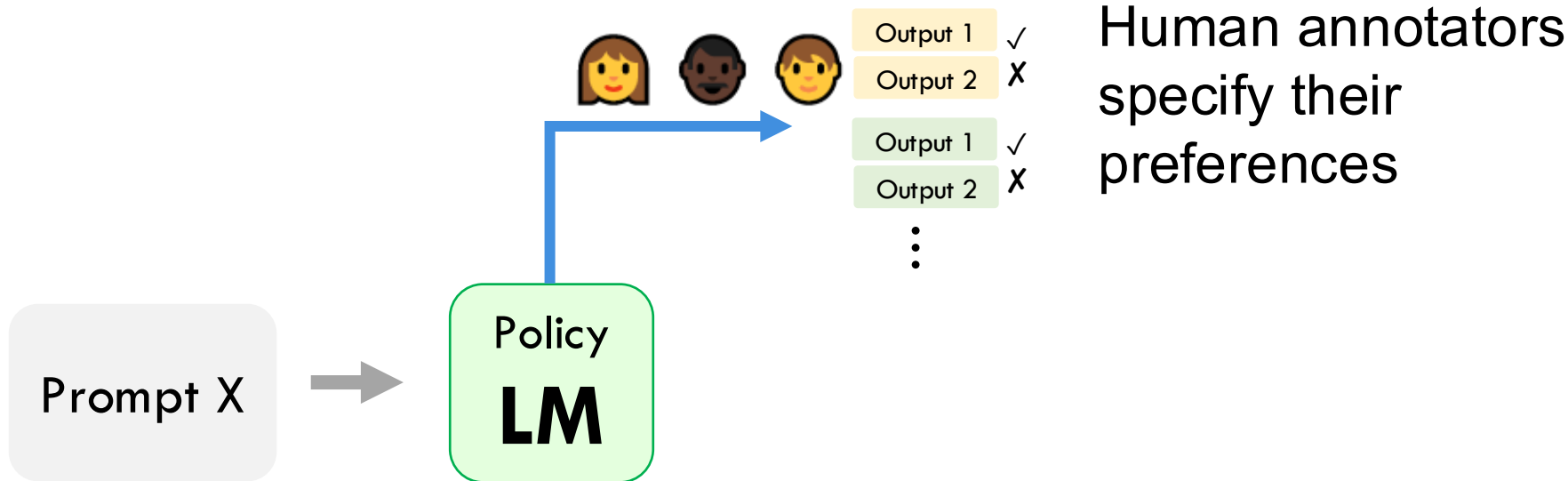
Step 2: Training the Policy

- Using policy gradients methods like (PPO), as discussed above.

Putting it Together



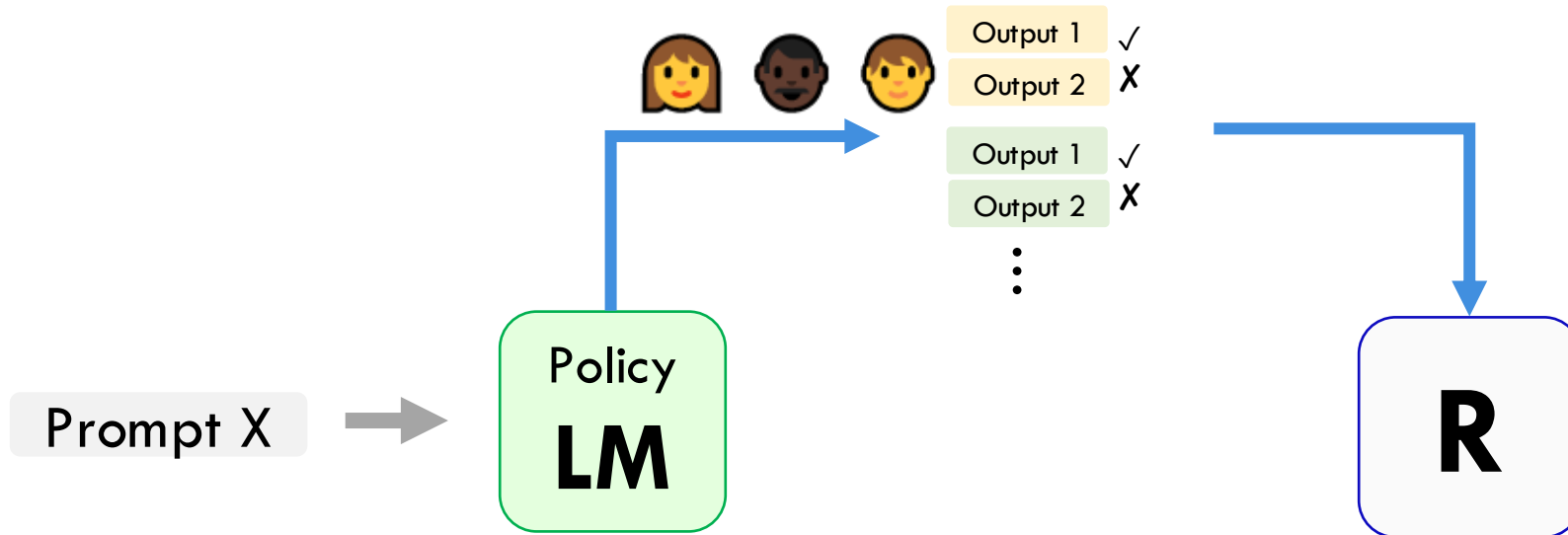
- First collect a dataset of preferences
 - Present multiple outputs to human annotators and ask them to rank the output based on preferability



Putting it Together (2)



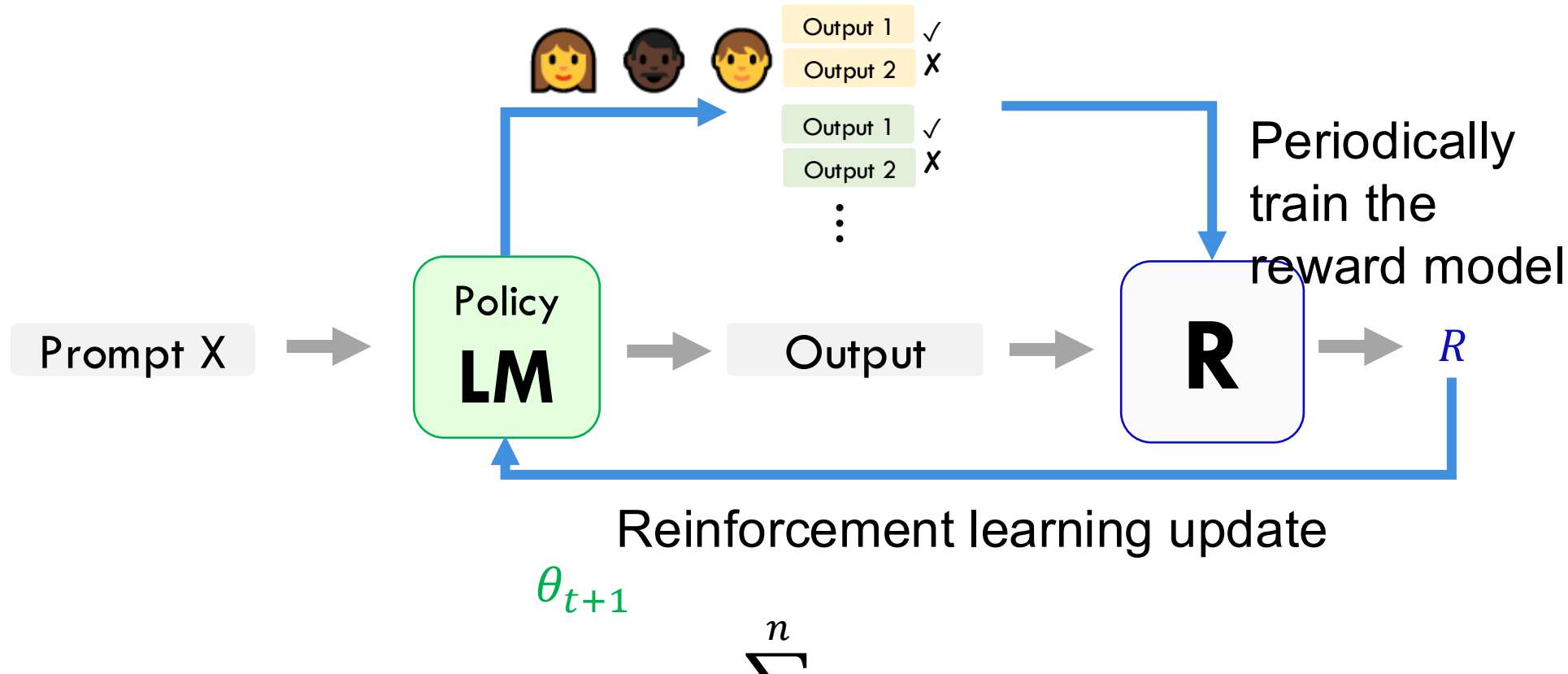
- Using this data, we can train a reward model
 - The reward model returns a scalar reward which should numerically represent the human preference.



Putting it Together (4)



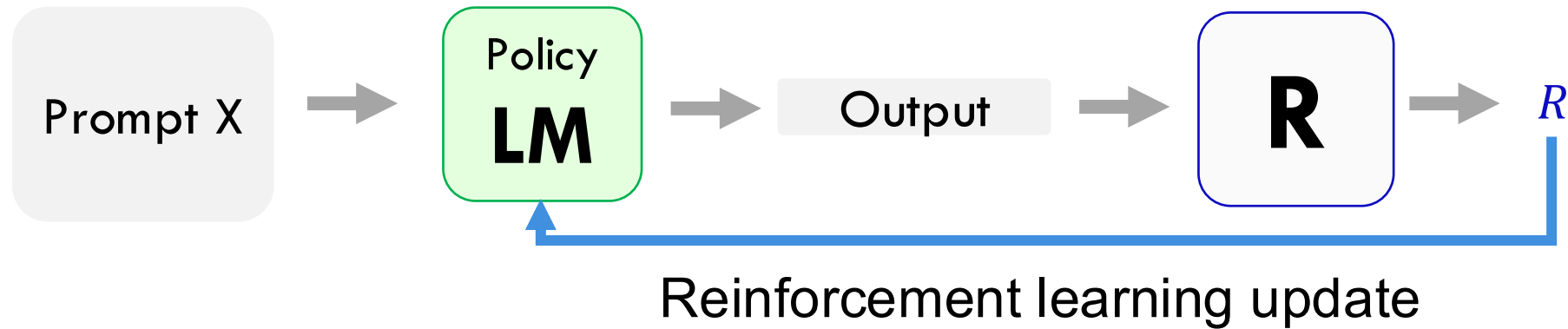
- Periodically train the reward model with more samples and human feedback



Putting it together (3)



- We want to learn a policy (a Language Model) that optimizes against the reward model



$$\theta_{t+1} \leftarrow \theta_t + \alpha \sum_{i=1}^n R(s_i; \mathbf{p}) \nabla_{\theta} \log p_{\theta}(s_i; \mathbf{p})$$

The overall recipe (for models like GPT 3.5) 🍳



Summary: RLHF / RLVR

- RL can help mitigate some of the problems with supervised instruction tuning
- RLHF uses two models
 - Reward model is trained via ranking feedback of humans.
 - Policy model learns to generate responses that maximize the reward model.
- Limitations:
 - RL can be tricky to get right
 - Training a good reward may require a lot of annotations